



DOCUMENTACIÓN TÉCNICA DE SOFTWARE

Sistema de Gestión Cafetería UIDE

Plataforma web de gestión de cafetería universitaria —
Backend Spring Boot & Frontend React

PROYECTO	GestionUIDE — Sistema de pedidos, facturación y administración para cafetería
COMPONENTES	API REST (Java / Spring Boot 2.x) + SPA (React + TypeScript + Vite + TailwindCSS)
DOCUMENTO	Documentación técnica, modelo de datos, casos de uso y manual de usuario
VERSIÓN	1.0
FECHA	Junio de 2026
ESTADO	Documento de referencia / línea base
CONFIDENCIALIDAD	Uso interno

Este documento describe el sistema *tal como está implementado* en el código fuente analizado. El análisis de mejoras y la auditoría de seguridad se entregan en un documento independiente.

Tabla de contenidos

1. Introducción y propósito

- 1.1 Objetivo del documento
- 1.2 Audiencia
- 1.3 Metodología documental

2. Alcance del sistema

- 2.1 Funcionalidades incluidas
- 2.2 Fuera de alcance

3. Glosario y definiciones

4. Arquitectura del sistema

- 4.1 Visión general
- 4.2 Stack tecnológico
- 4.3 Arquitectura en capas (backend)
- 4.4 Arquitectura del frontend
- 4.5 Seguridad y autenticación (flujo)

5. Modelo de datos

- 5.1 Entidades y relaciones
- 5.2 Diccionario de datos
- 5.3 Catálogos de estados (enums)

6. Roles y matriz de permisos

7. Catálogo de módulos funcionales

8. Casos de uso

- 7.1 Actores
- 7.2 Diagrama de casos de uso
- 7.3 Fichas detalladas

9. Diagramas de estados y flujos

10. Referencia de la API REST

11. Manual de usuario por rol

- 10.1 Cliente
- 10.2 Mesero
- 10.3 Cajero
- 10.4 Administrador

12. Requisitos, instalación y despliegue

13. Anexos

1. Introducción y propósito

El **Sistema de Gestión Cafetería UIDE** es una aplicación web que digitaliza la operación de una cafetería universitaria: publicación del menú, toma de pedidos, gestión de mesas y reservas, facturación, registro de pagos y administración de catálogo y usuarios. Está compuesto por una API REST construida con **Spring Boot** y una aplicación de página única (SPA) construida con **React + TypeScript**.

1.1 Objetivo del documento

Documentar el sistema *tal como está construido*: su arquitectura, su modelo de datos, sus reglas de negocio, sus servicios REST, los casos de uso soportados y el manual de operación para cada perfil de usuario. Sirve como línea base de conocimiento para el equipo de desarrollo, mantenimiento y para nuevos integrantes.

1.2 Audiencia

- **Equipo técnico** (desarrolladores backend/frontend, QA, DevOps): arquitectura, modelo de datos, API.
- **Usuarios funcionales** (administradores, cajeros, meseros, clientes): manual de usuario.
- **Docentes / evaluadores**: visión integral del producto, casos de uso y trazabilidad.

1.3 Metodología documental

La documentación combina prácticas de **UML** (actores y casos de uso, diagramas de estados) con elementos del estándar **IEEE 830 / ISO/IEC/IEEE 29148** para descripción de software (alcance, glosario, requisitos funcionales implícitos en módulos) y una **referencia de API** de estilo OpenAPI descriptivo. El manual de usuario sigue un enfoque orientado a tareas (paso a paso por rol).

2. Alcance del sistema

2.1 Funcionalidades incluidas

Área	Capacidades implementadas
Autenticación	Registro de clientes, inicio de sesión por usuario/email + contraseña, inicio de sesión con Google (OAuth2), recuperación de contraseña por código al correo, perfil del usuario autenticado.
Catálogo	Gestión de productos (CRUD, stock, imagen, disponibilidad) y categorías; menú público y menú para clientes; búsqueda por nombre y filtro por categoría.
Pedidos	Creación de pedidos (cliente o mesero), agregado/eliminación de ítems, ciclo de estados con validación de transiciones, descuento y restauración automática de stock.
Mesas	CRUD de mesas, control de estado (libre/ocupada/reservada) y consulta de disponibles.
Reservas	Solicitud de reserva por el cliente, confirmación y cancelación.
Facturación	Generación de factura desde un pedido o manual, numeración secuencial, cambio de estado, generación de PDF y envío por correo.
Pagos	Registro de pagos (efectivo, tarjeta, transferencia), cálculo de total pagado/pendiente y marcado de factura como pagada.
Favoritos	Marcar/desmarcar productos favoritos por cliente.
Administración	Gestión de usuarios y roles, panel (dashboard) con métricas: resumen, ventas, top de productos, stock bajo y ventas por mesero.

2.2 Fuera de alcance (no implementado en la base analizada)

Pasarela de pago real, facturación electrónica con el SRI, notificaciones en tiempo real a cocina, paginación en listados, tokens de refresco y verificación de correo en el registro. Estos puntos se desarrollan en el documento de análisis y mejoras.

3. Glosario y definiciones

Término	Definición
SPA	Single Page Application. Aplicación web que se carga una vez y actualiza la vista sin recargar la página.
API REST	Interfaz de servicios web basada en HTTP y JSON que expone las operaciones del backend.
JWT	JSON Web Token. Credencial firmada que transporta la identidad y los roles del usuario en cada petición.
DTO	Data Transfer Object. Objeto que transporta datos entre el cliente y el servidor, separado de la entidad de base de datos.
Entidad / Modelo	Clase Java mapeada a una tabla de base de datos mediante JPA/Hibernate.
WS (webservice)	Controlador REST del backend (capa de presentación de la API).
Rol	Perfil de usuario que determina los permisos: ADMIN, MESERO, CAJERO o CLIENTE.
BCrypt	Algoritmo de hash con sal usado para almacenar contraseñas de forma segura.
IVA	Impuesto al Valor Agregado aplicado al subtotal de pedidos y facturas.

4. Arquitectura del sistema

4.1 Visión general

El sistema sigue una arquitectura **cliente-servidor desacoplada**. El navegador ejecuta la SPA de React, que consume la API REST de Spring Boot mediante peticiones HTTP con JSON. La autenticación es **sin estado (stateless)** basada en JWT: tras el login, el servidor emite un token que el cliente envía en la cabecera `Authorization: Bearer <token>` de cada petición protegida. La persistencia se realiza sobre **MySQL** a través de Spring Data JPA / Hibernate.



4.2 Stack tecnológico

Capa	Tecnologías
Frontend	React, TypeScript, Vite, React Router, TailwindCSS, Axios, lucide-react (iconos), react-hot-toast (notificaciones), Context API (estado de autenticación, carrito y tema).
Backend	Java, Spring Boot 2.x, Spring Web (REST), Spring Security, Spring Data JPA / Hibernate, Bean Validation, Lombok, jjwt (JWT), BCrypt, JavaMailSender (SMTP), librería de generación de PDF de facturas.
Base de datos	MySQL (esquema gestion), generación de esquema automática por Hibernate.
Integraciones	Google Identity (verificación de <i>id token</i> del lado del servidor), correo SMTP (Gmail) para bienvenida, recuperación de contraseña y envío de facturas.

4.3 Arquitectura en capas (backend)

El backend está organizado en paquetes que reflejan una arquitectura en capas clásica, lo que favorece la separación de responsabilidades:

Presentación — webservice/

Controladores REST (@RestController): AutenticacionWS, ProductoWS, CategoryWS, PedidoWS, FacturaWS, PagoWS, MesaWS, ReservaWS, FavoritosWS, UsuarioWS, DashboardWS, EmpresaWS. Validan la entrada y devuelven un sobre uniforme ApiResponse.

Transferencia — dto/

Objetos de entrada/salida con validaciones (@NotBlank, @Email, @Size, @Positive): LoginDto, UsuarioRegistroDto, ProductoDto, CrearPedidoDto, PagoDto, FacturaDto, etc.

Lógica de negocio — service/

Reglas del dominio (@Service @Transactional): UsuarioService, ProductoService, PedidoService, FacturaService, PagoService, ReservaService, DashboardService, EmailService, FacturaPdfService, GoogleIdTokenService, RolService, etc.

Persistencia — repository/ + model/

Interfaces Spring Data JPA y entidades mapeadas (Usuario, Roles, Producto, Category, Mesa, Pedido, DetallePedido, Factura, DetalleFactura, Pago, Reserva, Favoritos, Empresa).

Transversales — exception/, enums/, JWTAuthorizationFilter

Manejo global de excepciones (@RestControllerAdvice), catálogos de estados y filtro de autorización JWT.

4.4 Arquitectura del frontend

La SPA organiza el código por responsabilidad: `api/` (cliente Axios con interceptores), `services/` (un servicio por recurso que mapea los endpoints), `context/` (estado global de autenticación, carrito y tema), `routes/` (enrutamiento con guardias por rol), `components/` (UI y layout), `pages/` (vistas agrupadas por rol) y `utils/ + types/`. El interceptor de Axios adjunta automáticamente el token y, ante una respuesta 401, cierra la sesión y redirige al login.

Elemento	Rol
AuthContext	Mantiene el usuario y el token (en localStorage), expone login, registro, logout, hasRole y el rol principal.

CartContext	Carrito de compras del cliente: ítems, subtotal, IVA (15%), total y contador.
ProtectedRoute / RoleRoute	Guardias de navegación: exigen sesión y, opcionalmente, un rol; redirigen al inicio del rol correspondiente.
DashboardLayout	Marco con barra lateral y superior que adapta el menú según el rol activo.

4.5 Seguridad y autenticación (flujo)



Nota de implementación

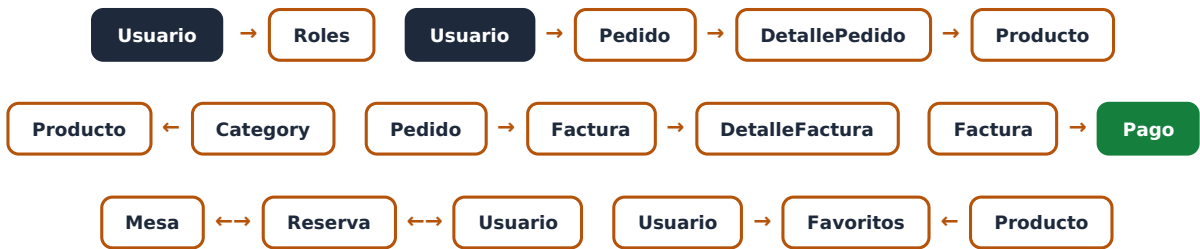
Las contraseñas se almacenan con **BCrypt** (hash con sal). El campo `password` está anotado con `@JsonIgnore`, por lo que nunca se serializa en las respuestas. La recuperación de contraseña no revela si un correo existe. Las observaciones sobre el endurecimiento de esta capa se detallan en el documento de auditoría.

5. Modelo de datos

5.1 Entidades y relaciones

El dominio se compone de 13 entidades. Las relaciones principales son:

- **Usuario** 1—N **Roles** (un usuario puede tener varios roles).
- **Category** 1—N **Producto**.
- **Pedido** N—1 **Mesa**, N—1 **Usuario** (cliente) y N—1 **Usuario** (mesero); 1—N **DetallePedido**; 1—1 **Factura**.
- **DetallePedido** N—1 **Producto**.
- **Factura** N—1 **Usuario** (cliente y cajero), N—1 **Empresa**; 1—N **DetalleFactura** y 1—N **Pago**.
- **Reserva** N—1 **Usuario** y N—1 **Mesa**.
- **Favoritos** N—1 **Usuario** y N—1 **Producto**.



5.2 Diccionario de datos (entidades principales)

Entidad	Campos clave	Descripción
Usuario	id, username (único), email (único), password (hash, oculto), nombre, apellido, telefono, fotoUrl, activo, fechaRegistro, fechaActualizacion	Persona que accede al sistema. Indexado por username y email.
Roles	id, usuario, rol (enum)	Asignación de un rol a un usuario (relación con la entidad Usuario).
Category	categoryId, name (único), descripcion, activo	Categoría de productos del menú.
Producto	id, nombre, descripcion, precio, stock, disponible, imagenUrl, category	Artículo del menú con control de inventario y disponibilidad.
Mesa	id, numeroMesa, capacidad, estado (enum), ubicacion, activo, fechaCreacion	Mesa física del local.
Pedido	id, mesa, cliente, mesero, estado (enum), subtotal, iva, total, notas, fechaPedido	Comanda con sus detalles; máquina de estados con transiciones validadas.
DetallePedido	id, pedido, producto, cantidad, precioUnitario, subtotal, notas	Línea de un pedido.
Factura	id, numeroFactura (FAC-NNNNNN), subtotal, iva, descuento, total, estado (enum), cliente, cajero, empresa, pedido	Comprobante de venta. Numeración secuencial única.
DetalleFactura	id, factura, producto, cantidad, subtotal	Línea de una factura.
Pago	id, factura, metodoPago (enum), monto, referencia, fechaPago	Pago aplicado a una factura. Una factura admite varios pagos.
Reserva	id, usuario, mesa, fechaReserva, duracionHoras, numPersonas, estado (enum), notas, fechaCreacion	Reserva de mesa solicitada por un cliente.
Favoritos	id, usuario, producto	Producto marcado como favorito por un cliente.
Empresa	datos del emisor (incl. porcentaje de IVA)	Datos del negocio usados en la facturación manual.

5.3 Catálogos de estados (enums)

Enum	Valores
Rol	ADMIN, CAJERO, MESERO, CLIENTE
EstadoPedido	PENDIENTE, EN_PREPARACION, SERVIDO, PAGADO, CANCELADO

EstadoFactura	PENDIENTE, PAGADA, CANCELADA
EstadoMesa	LIBRE, OCUPADA, RESERVADA
EstadoReserva	PENDIENTE, CONFIRMADA, CANCELADA, COMPLETADA
MetodoPago	EFFECTIVO, TARJETA, TRANSFERENCIA

6. Roles y matriz de permisos

El sistema define cuatro roles. La interfaz adapta el menú y las vistas según el rol; cada rol tiene una pantalla de inicio propia.

Rol	Inicio	Responsabilidad
ADMIN	Panel de administración	Configura el catálogo, mesas y usuarios; supervisa pedidos, facturas y métricas del negocio.
MESERO	Pedidos	Gestiona mesas, crea y atiende pedidos, actualiza su estado.
CAJERO	Facturas	Factura pedidos servidos, emite facturas manuales y registra pagos.
CLIENTE	Menú	Explora el menú, arma el carrito, realiza pedidos, gestiona favoritos y reservas.

Matriz funcional (vista de interfaz)

Funcionalidad	Admin	Mesero	Cajero	Cliente
Ver menú / productos	✓	✓	✓	✓
CRUD de productos y categorías	✓	-	-	-
Gestión de mesas	✓	✓*	-	-
Crear pedido	✓	✓	-	✓
Cambiar estado de pedido	✓	✓	✓**	-
Emitir factura / registrar pago	✓	-	✓	-
Gestión de usuarios y roles	✓	-	-	-
Favoritos y reservas	-	-	-	✓
Panel de métricas	✓	resumen	resumen	-

* El mesero cambia el estado de las mesas que atiende. ** El cajero marca el pedido como pagado al facturar. Esta matriz refleja la *experiencia de la interfaz*; la verificación de estos permisos en el servidor se evalúa en el documento de auditoría.

7. Catálogo de módulos funcionales

Módulo	Pantallas (frontend)	Servicios (backend)
Autenticación	LoginPage, AuthModal	AutenticacionWS, UsuarioService, GoogleIdTokenService, EmailService
Catálogo	PublicMenuPage, ClienteMenu, AdminProductos, AdminCategorías	ProductoWS, CategoryWS
Pedidos	MeseroNuevoPedido, MeseroPedidos, AdminPedidos, ClientePedidos, CheckoutPage, CartDrawer	PedidoWS, PedidoService, DetallePedidoService
Mesas	AdminMesas, MeseroMesas	MesaWS, MesaService
Reservas	ClienteReservas	ReservaWS, ReservaService
Facturación	CajeroFacturas, CajeroFacturaManual, CajeroPedidosServidos, AdminFacturas	FacturaWS, FacturaService, FacturaPdfService
Pagos	(integrado en facturación)	PagoWS, PagoService
Favoritos	ClienteFavoritos	FavoritosWS, FavoritoService
Usuarios	AdminUsuarios, PerfilPage	UsuarioWS, RolService
Métricas	AdminDashboard, MeseroDashboard, CajeroDashboard	DashboardWS, DashboardService

8. Casos de uso

8.1 Actores

- **Cliente** — consume el servicio: pedidos, favoritos, reservas.
- **Mesero** — atiende mesas y gestiona pedidos.
- **Cajero** — factura y cobra.
- **Administrador** — configura y supervisa el sistema.
- **Sistemas externos** — Google (OAuth2) y servidor SMTP (correo).

8.2 Diagrama de casos de uso (resumen por actor)

Actor	Casos de uso
Cliente	Registrarse · Iniciar sesión (local / Google) · Recuperar contraseña · Ver menú · Agregar al carrito · Realizar pedido · Ver mis pedidos · Gestionar favoritos · Solicitar reserva · Editar perfil
Mesero	Ver mesas · Crear pedido · Agregar/quitar ítems · Avanzar estado del pedido · Ver pedidos · Liberar mesa
Cajero	Ver pedidos servidos · Generar factura desde pedido · Crear factura manual · Registrar pago · Descargar PDF · Enviar factura por correo
Administrador	CRUD productos/categorías · CRUD mesas · Gestionar usuarios y roles · Supervisar pedidos y facturas · Consultar métricas

8.3 Fichas de casos de uso detalladas

CU-01 — Realizar pedido (Cliente)

ACTOR	Cliente autenticado
PRECONDICIÓN	El cliente ha iniciado sesión y existen productos disponibles.
FLUJO PRINCIPAL	1) El cliente explora el menú y agrega productos al carrito (cantidad y notas). 2) Abre el carrito; el sistema calcula subtotal, IVA (15%) y total. 3) Procede al checkout. 4) Confirma el pedido. 5) El sistema valida stock, crea el pedido en estado PENDIENTE, descuenta el stock y vacía el carrito. 6) Se muestra confirmación.
FLUJO ALTERNO	5a) Si no hay stock suficiente, el sistema informa el producto y la cantidad disponible y cancela la creación.
POSTCONDICIÓN	Pedido registrado y visible en “Mis pedidos”; stock actualizado.

CU-02 — Atender y avanzar un pedido (Mesero)

ACTOR	Mesero
PRECONDICIÓN	Existe una mesa disponible o reservada.
FLUJO PRINCIPAL	1) El mesero crea un pedido seleccionando mesa y productos. 2) La mesa pasa a OCUPADA. 3) El mesero avanza el estado: PENDIENTE → EN_PREPARACION → SERVIDO. 4) Puede agregar o quitar ítems mientras el pedido esté PENDIENTE o EN_PREPARACION (con ajuste de stock).
FLUJO ALTERNO	3a) Cancelar el pedido restaura el stock y libera la mesa. No se puede cancelar un pedido ya PAGADO.
POSTCONDICIÓN	Pedido en estado SERVIDO, listo para facturar.

CU-03 — Facturar y cobrar (Cajero)

ACTOR	Cajero
PRECONDICIÓN	Existe un pedido en estado SERVIDO (o se factura de forma manual).
FLUJO PRINCIPAL	1) El cajero selecciona un pedido servido y genera la factura (numeración FAC-NNNNNN). 2) Registra uno o varios pagos (efectivo/tarjeta/transferencia). 3) Cuando el total pagado cubre el total, la factura pasa a PAGADA y el pedido a PAGADO; la mesa se libera. 4) Puede descargar el PDF o enviarlo por correo.
FLUJO ALTERNO	1a) Factura manual: el cajero arma ítems libres con datos de empresa; el sistema calcula IVA y descuento.
POSTCONDICIÓN	Factura emitida y, si se cobró el total, marcada como PAGADA.

CU-04 — Gestionar usuarios y roles (Administrador)

ACTOR	Administrador
FLUJO PRINCIPAL	1) El administrador lista los usuarios. 2) Crea un usuario asignándole un rol. 3) Activa/desactiva cuentas y agrega o quita roles. 4) Edita datos de perfil.
POSTCONDICIÓN	Usuarios y roles actualizados.

9. Diagramas de estados y flujos

9.1 Ciclo de vida del pedido



desde PENDIENTE / EN_PREPARACION / SERVIDO → CANCELADO (restaura stock y libera mesa)

Las transiciones están validadas en el servidor: solo se permite avanzar al estado inmediato siguiente. Un pedido PAGADO no puede cancelarse.

9.2 Ciclo de vida de la factura



9.3 Ciclo de vida de la mesa



9.4 Ciclo de vida de la reserva



10. Referencia de la API REST

Base: /api. Salvo los recursos públicos, todas las peticiones requieren la cabecera Authorization: Bearer <token>. Las respuestas usan el sobre uniforme { success, message, data }.

Autenticación — /api/auth público

Método	Ruta	Descripción
POST	/login	Inicia sesión con usuario/email y contraseña; devuelve token y datos.
POST	/registro	Registra un cliente y envía correo de bienvenida.
POST	/oauth2/google	Inicia sesión / registra mediante <i>id token</i> de Google.
POST	/recuperar-password	Envía un código de recuperación al correo.
POST	/verificar-codigo	Valida el código y cambia la contraseña.
GET	/verificar/username/{u} · /verificar/email	Comprueba disponibilidad de username/email.
GET	/me	Devuelve el usuario autenticado.
POST	/logout	Cierre de sesión (del lado del cliente).

Productos — /api/productos Categorías — /api/categorias GET público

Método	Ruta	Descripción
GET	/productos · /disponibles · /{id}	Listar, disponibles, obtener por id.
GET	/productos/categoria/{id} · /buscar? nombre=	Filtrar por categoría y buscar por nombre.
POST/PUT/DELETE	/productos · /{id}	Crear, actualizar y eliminar producto.
PUT/POST	/{id}/stock/agregar · /desactivar · /activar · /imagen	Ajuste de stock, disponibilidad y subida de imagen.
GET...DELETE	/categorias · /activas · /{id}	CRUD de categorías y listado de activas.

Pedidos — /api/pedidos

Método	Ruta	Descripción
GET	/pedidos · /{id} · /pendientes	Listar, obtener, pendientes.
GET	/cliente/{id} · /mesero/{id} · /mesa/{id} · /estado/{e}	Filtros por cliente, mesero, mesa y estado.
POST/PUT	/pedidos · /{id}/estado?nuevoEstado=	Crear pedido y cambiar estado.
POST/DELETE	/{id}/items · /{pid}/items/{iid}	Agregar/eliminar ítems.

Facturas — /api/facturas Pagos — /api/pagos

Método	Ruta	Descripción
GET	/facturas · /{id} · /cliente/{id} · /cajero/{id} · /estado/{e} · /numero/{n}	Consultas de facturas.
POST	/desde-pedido/{pedidoId}?cajeroId= · /manual	Generar desde pedido o factura manual.
PUT/GET/POST	/{id}/estado · /{id}/pdf · /{id}/enviar-email? email=	Estado, descarga PDF, envío por correo.
POST	/pagos · /pagos/efectivo · /pagos/tarjeta	Registrar pago.
GET	/pagos/factura/{id} · /total · /pendiente · /pagada	Pagos y saldos de una factura.

Mesas, Reservas, Favoritos, Usuarios, Dashboard

--

Recurso	Endpoints principales
Mesas	/api/mesas (CRUD), /disponibles, /{id}/estado?estado=
Reservas	/api/reservas, /usuario/{id}, POST ?usuarioId=, /{id}/confirmar, /{id}/cancelar
Favoritos	/api/favoritos/usuario/{id}, POST, /toggle, /existe/{prodId}?usuarioId=, DELETE /{id}
Usuarios	/api/usuarios (CRUD), /activos, /rol/{rol}, /{id}/rol, /activar, /desactivar, /password, /foto, /perfil
Dashboard	/api/dashboard/resumen, /ventas, /top-productos, /stock-bajo, /ventas-por-mesero

11. Manual de usuario por rol

11.1 Cliente

1. **Crear cuenta / iniciar sesión.** Desde la página inicial, regístrate con tus datos o entra con Google. Si olvidaste tu contraseña, usa “recuperar” y sigue el código enviado a tu correo.
2. **Explorar el menú.** Navega por categorías o busca por nombre. Marca productos como favoritos con el corazón.
3. **Armar el carrito.** Agrega productos indicando cantidad y notas. El carrito muestra subtotal, IVA y total.
4. **Realizar el pedido.** Ve al checkout y confirma. Si algún producto no tiene stock, el sistema te avisa.
5. **Seguimiento.** Revisa el estado en “Mis pedidos”. Gestiona tus reservas en la sección correspondiente.
6. **Perfil.** Actualiza tus datos y foto desde “Mi perfil”.

11.2 Mesero

1. **Ver mesas.** Consulta el estado de cada mesa (libre, ocupada, reservada).
2. **Nuevo pedido.** Selecciona la mesa y agrega productos; al confirmar, la mesa pasa a ocupada.
3. **Gestionar el pedido.** Avanza el estado del pedido a medida que se prepara y se sirve. Mientras esté pendiente o en preparación puedes agregar o quitar ítems.
4. **Cancelar.** Si procede, cancela el pedido: el stock se restaura y la mesa se libera.

11.3 Cajero

1. **Pedidos servidos.** Abre la lista de pedidos listos para cobrar.
2. **Facturar.** Genera la factura desde el pedido, o crea una factura manual con ítems libres.
3. **Registrar el pago.** Selecciona el método (efectivo, tarjeta o transferencia) e ingresa el monto. Cuando se cubre el total, la factura queda como pagada.
4. **Entregar comprobante.** Descarga el PDF o envíalo al correo del cliente.

11.4 Administrador

1. **Catálogo.** Crea y edita productos y categorías; ajusta stock, precios, imágenes y disponibilidad.
2. **Mesas.** Define mesas (número, capacidad, ubicación) y su estado.
3. **Usuarios.** Crea cuentas, asigna roles, activa/desactiva usuarios.
4. **Supervisión.** Consulta pedidos y facturas de todo el local.
5. **Métricas.** Revisa el panel: ventas, productos más vendidos, stock bajo y rendimiento por mesero.

12. Requisitos, instalación y despliegue

12.1 Requisitos

- **Backend:** JDK compatible con Spring Boot 2.x, Maven/Gradle, MySQL 8.
- **Frontend:** Node.js + gestor de paquetes (npm/pnpm/yarn), Vite.
- **Servicios:** cuenta SMTP para correo, credenciales de Google OAuth (Client ID).

12.2 Configuración (resumen de `application.properties`)

```
# Servidor y base de datos
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/gestion
spring.jpa.hibernate.ddl-auto=update

# Integraciones
jwt.secret=<clave de firma JWT>
google.oauth.client-id=<client id>
spring.mail.host=smtp.gmail.com spring.mail.port=587

# Límites de subida
spring.servlet.multipart.max-file-size=5MB
```

Importante

El archivo de configuración analizado contiene credenciales en texto plano (base de datos, clave JWT, contraseña de correo). Antes de cualquier despliegue deben externalizarse a variables de entorno o a un gestor de secretos. El detalle y la remediación están en el documento de auditoría.

12.3 Pasos de despliegue (referencia)

1. Crear la base de datos `gestion` en MySQL.
2. Configurar las variables/propiedades y arrancar el backend (`mvn spring-boot:run` o el JAR empaquetado). Al iniciar se crean las carpetas `uploads/productos` y `uploads/usuarios`.
3. Configurar `VITE_API_URL` apuntando a la API y construir el frontend (`npm run build`); servir los estáticos detrás de un proxy/web server.

13. Anexos

A. Sobre de respuesta estándar

```
{ "success": true, "message": "Login correcto", "data": { ... } }
```

B. Convenciones

- Numeración de facturas: FAC-NNNNNN (secuencial, único).
- IVA aplicado: 15% sobre el subtotal (configurable por empresa en facturación manual).
- Imágenes y fotos se sirven bajo la ruta pública `/uploads/**`.

C. Mapa de archivos del frontend

`api/` cliente HTTP · `services/` 11 servicios por recurso · `context/` Auth, Cart, Theme · `routes/` enrutamiento y guardias · `pages/` vistas por rol (admin, mesero, cajero, cliente, auth, common) · `components/` UI y layout · `utils/` formato y roles · `types/` tipos compartidos.